USER INTERFACE DESIGN EXP:3 COMMAND INTERFACES CLI, GUI, VUI

SANSHIA.J 240701471

Aim

To design and implement an Alert System application using three different interfaces:

- Command Line Interface (CLI)
- Graphical User Interface (GUI)
- Voice User Interface (VUI)

and compare their usability and user satisfaction.

Objective

- To understand different types of user interfaces.
- To implement task management using CLI, GUI and VUI.
- To compare efficiency, usability and user satisfaction.
- To analyze advantages and limitations of each interface type.

Tools Used

- Python Programming Language
- Tkinter (for GUI)
- SpeechRecognition Library (for VUI)
- PyAudio (Microphone input)
- VS Code / Terminal

Theory

User Interfaces allow interaction between users and systems. There are multiple types of interfaces:

1. CLI (Command Line Interface) – Text-based interaction.
2. GUI (Graphical User Interface) – Visual interaction using buttons and windows.
3. VUI (Voice User Interface) – Interaction using speech commands.

Each interface provides different levels of usability, learning curve and efficiency.

This project demonstrates how the same application (To-Do List Manager) can be implemented using different interaction styles.

System Description

The To-Do List Manager allows users to:

- Add new alerts
- View Active alerts
- Remove alerts
- Receive notification messages
- Navigate back to main menu

The application has a main menu screen where the user selects:

- CLI
- GUI
- VUI (Voice)
- Exit

A. CLI (Command Line Interface)

Description

The CLI version allows users to manage tasks by typing commands in a text-based terminal window.

The interface displays menu options:

1. Add alert
2. Remove alert
3. Show alert
4. Back to Main Menu

The user enters the corresponding number to perform operations.

How It Works

- The program stores alerts in a list.
- User inputs are read using `input()`.
- Based on the selection, alerts are added, removed, or displayed.
- Error handling prevents invalid selections.

Code

alert_cli.py > ...

```python
import sys
import datetime

def send_alert(message):
    timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    alert_entry = f"[{timestamp}] ALERT: {message}\n"

    try:
        with open("alerts_log.txt", "a") as file:
            file.write(alert_entry)

        print("\n✅ Emergency Alert Sent Successfully!")
        print(f"Message: {message}")
        print(f"Time: {timestamp}")

    except Exception as e:
        print(f"\n❌ Error sending alert: {e}")


def show_alerts():
    try:
        with open("alerts_log.txt", "r") as file:
            alerts = file.read()

        if alerts.strip() == "":
            print("\n📭 No alerts found.")
        else:
            print("\n📜 Alert History:")
            print(alerts)

    except FileNotFoundError:
        print("\n📭 No alerts have been recorded yet.")


def search_alerts(date=None, time=None):
    try:
        with open("alerts_log.txt", "r") as file:
            lines = file.readlines()

        results = []

        for line in lines:
            if date and date not in line:
                continue
            if time and time not in line:
                continue
            results.append(line.strip())
```

```python
def search_alerts(date=None, time=None):

        if results:
            print("\n🔎 Matching Alerts:")
            for r in results:
                print(r)
        else:
            print("\n❌ No alerts found for given date/time.")

    except FileNotFoundError:
        print("\n📭 No alerts have been recorded yet.")


if __name__ == "__main__":
    if len(sys.argv) < 2:
        print("\nUsage:")
        print("Send Alert : py alert_cli.py send \"message\"")
        print("View Alerts: py alert_cli.py view")
        print("Search Date & Time: py alert_cli.py search YYYY-MM-DD HH:MM")
        print("Search Date only  : py alert_cli.py search YYYY-MM-DD")
        print("Search Time only  : py alert_cli.py search HH:MM")

    elif sys.argv[1].lower() == "send":
        if len(sys.argv) < 3:
            print("\n⚠️ Please provide an emergency message.")
        else:
            message = " ".join(sys.argv[2:])
            send_alert(message)

    elif sys.argv[1].lower() == "view":
        show_alerts()

    elif sys.argv[1].lower() == "search":
        if len(sys.argv) == 3:
            query = sys.argv[2]
            if ":" in query:
                search_alerts(time=query)
            else:
                search_alerts(date=query)

        elif len(sys.argv) >= 4:
            search_alerts(date=sys.argv[2], time=sys.argv[3])

        else:
            print("\n⚠️ Please provide date and/or time.")

    else:
        print("\n❌ Invalid command.")
```

B. GUI (Graphical User Interface)

Description

The GUI version uses Tkinter to create a window-based interface.

Features:

- Entry field to type task
- Add button
- Delete button
- Clear All button
- Task display area
- Back to Main button

The GUI provides a user-friendly and interactive environment.

How It Works

- Tkinter window is created using `Tk()`
- Buttons trigger functions using `command=`
- Tasks are displayed inside a listbox or text area
- Visual feedback improves usability

The GUI layout includes:

- Title: GUI Mode
- Task input field
- Scrollable task display
- Control buttons at botton

Code

alert_gui.py > send_alert

```python
import tkinter as tk
from tkinter import messagebox, scrolledtext
import datetime
import os

LOG_FILE = "alerts_log.txt"


def send_alert():
    message = message_entry.get()

    if not message.strip():
        messagebox.showwarning("Warning", "Please enter an emergency message.")
        return

    timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    alert_entry = f"[{timestamp}] ALERT: {message}\n"

    with open(LOG_FILE, "a") as file:
        file.write(alert_entry)

    messagebox.showinfo("Success", "🚨 Emergency alert sent successfully!")
    message_entry.delete(0, tk.END)


def view_alerts():
    output_area.delete(1.0, tk.END)

    if not os.path.exists(LOG_FILE):
        output_area.insert(tk.END, "No alerts recorded yet.")
        return

    with open(LOG_FILE, "r") as file:
        output_area.insert(tk.END, file.read())


def search_alerts():
    date = date_entry.get()
    time = time_entry.get()

    output_area.delete(1.0, tk.END)

    if not os.path.exists(LOG_FILE):
        output_area.insert(tk.END, "No alerts recorded yet.")
        return

    with open(LOG_FILE, "r") as file:
        lines = file.readlines()
```

alert_gui.py > ...

```python
def search_alerts():
    results = []
    for line in lines:
        if date and date not in line:
            continue
        if time and time not in line:
            continue
        results.append(line)

    if results:
        for r in results:
            output_area.insert(tk.END, r)
    else:
        output_area.insert(tk.END, "No matching alerts found.")



root = tk.Tk()
root.title("Smart Emergency Alert System")
root.geometry("750x580")
root.resizable(False, False)
root.configure(bg="#121212")

tk.Label(
    root,
    text="🚨 Smart Emergency Alert System",
    font=("Segoe UI", 20, "bold"),
    bg="#121212",
    fg="#ffffff"
).pack(pady=15)

card1 = tk.Frame(root, bg="#1e1e1e")
card1.pack(padx=25, pady=10, fill="x")

tk.Label(
    card1,
    text="Emergency Message",
    font=("Segoe UI", 12, "bold"),
    bg="#1e1e1e",
    fg="#ffffff"
).pack(anchor="w", padx=15, pady=(12, 5))

message_entry = tk.Entry(
    card1,
    font=("Segoe UI", 11),
    bg="#2b2b2b",
    fg="white",
    insertbackground="white",
```

```python
tk.Button(
    card1,
    text="SEND ALERT",
    font=("Segoe UI", 11, "bold"),
    bg="#e53935",
    fg="white",
    activebackground="#c62828",
    relief="flat",
    padx=25,
    pady=10,
    command=send_alert
).pack(pady=12)

card2 = tk.Frame(root, bg="#1e1e1e")
card2.pack(padx=25, pady=10, fill="x")

tk.Label(
    card2,
    text="Search Alerts",
    font=("Segoe UI", 12, "bold"),
    bg="#1e1e1e",
    fg="#ffffff"
).pack(anchor="w", padx=15, pady=(12, 5))

search_frame = tk.Frame(card2, bg="#1e1e1e")
search_frame.pack(padx=15, pady=8)

def dark_entry(parent, width):
    return tk.Entry(
        parent,
        width=width,
        bg="#2b2b2b",
        fg="white",
        insertbackground="white",
        bd=0
    )

tk.Label(search_frame, text="Date (YYYY-MM-DD)", bg="#1e1e1e", fg="white").grid(row=0, column=0, padx=6)
date_entry = dark_entry(search_frame, 15)
date_entry.grid(row=0, column=1, padx=6)

tk.Label(search_frame, text="Time (HH:MM)", bg="#1e1e1e", fg="white").grid(row=0, column=2, padx=6)
time_entry = dark_entry(search_frame, 10)
time_entry.grid(row=0, column=3, padx=6)

btn_frame = tk.Frame(card2, bg="#1e1e1e")
btn_frame.pack(pady=10)
```

```python
tk.Button(
    btn_frame,
    text="Search",
    bg="#1e88e5",
    fg="white",
    relief="flat",
    width=16,
    pady=8,
    command=search_alerts
).grid(row=0, column=0, padx=6)

tk.Button(
    btn_frame,
    text="View All Alerts",
    bg="#546e7a",
    fg="white",
    relief="flat",
    width=16,
    pady=8,
    command=view_alerts
).grid(row=0, column=1, padx=6)

output_area = scrolledtext.ScrolledText(
    root,
    width=85,
    height=12,
    bg="#0f0f0f",
    fg="#00ff9c",
    insertbackground="white",
    font=("Consolas", 10),
    bd=0
)
output_area.pack(padx=25, pady=15)

root.mainloop()
```

C. VUI (Voice User Interface)

Description

The VUI version allows users to manage tasks using voice commands.

Example Commands:

- "Add homework"
- "Remove homework"
- "Show tasks"
- "Back"

The interface shows:

- Voice Ready status
- Speak button
- Response display area
- Task count

How It Works

- Uses SpeechRecognition library
- Captures voice input using microphone
- Converts speech to text using Google Speech API
- Processes commands and updates task list
- Displays response on screen

Installation Required:

```
Code

pip install SpeechRecognition
pip install PyAudio
```

Code

alert_vui.py > ...

```python
import speech_recognition as sr
import datetime
import os

LOG_FILE = "alerts_log.txt"

def log_alert(message):
    timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    with open(LOG_FILE, "a") as f:
        f.write(f"[{timestamp}] ALERT: {message}\n")
    print(f"✅ Alert logged: {message}")

def search_alerts(keyword):
    if not os.path.exists(LOG_FILE):
        print("No alerts recorded yet.")
        return

    with open(LOG_FILE, "r") as f:
        lines = f.readlines()

    found = False
    for line in lines:
        if keyword in line:
            print(line.strip())
            found = True

    if not found:
        print("❌ No matching alerts found.")

def process_command(command):
    command = command.lower()
    print(f"🎙 You said: {command}")

    if command.startswith("send alert"):
        message = command.replace("send alert", "").strip()
        if message:
            log_alert(message)
        else:
            print("❌ No emergency message detected.")

    elif command.startswith("search alerts"):
        keyword = command.replace("search alerts", "").strip()
        if keyword:
            search_alerts(keyword)
        else:
            print("❌ Please specify date or time.")

    else:
```

```python
    else:
        print("❌ Command not recognized.")

def listen():
    recognizer = sr.Recognizer()
    mic = sr.Microphone()

    print("🎙 Listening... (say a command)")

    with mic as source:
        recognizer.adjust_for_ambient_noise(source)
        audio = recognizer.listen(source)

    try:
        command = recognizer.recognize_google(audio)
        process_command(command)

    except sr.UnknownValueError:
        print("❌ Sorry, could not understand.")
    except sr.RequestError:
        print("❌ Speech service unavailable.")

if __name__ == "__main__":
    listen()
```

User Satisfaction Comparison

Based on implementation and testing:

1. Efficiency (Time to Complete Task)

- CLI: Fastest for experienced users.
- GUI: Moderate speed, visually guided.
- VUI: Depends on speech recognition accuracy.

2. Learning Curve

- CLI: High learning curve.
- GUI: Easy for beginners.
- VUI: Very easy but depends on clarity of speech.

3. Error Handling

- CLI: Limited feedback.
- GUI: Clear error messages.
- VUI: May misinterpret voice commands.

4. User Satisfaction

- CLI: Preferred by technical users.
- GUI: Most user-friendly and widely accepted.
- VUI: Innovative and hands-free, but accuracy dependent.

Advantages

- Demonstrates multiple interface types.
- Enhances understanding of HCI concepts.
- Shows practical comparison of interaction styles.
- Improves programming and UI design skills.

Limitations

- VUI depends on internet and microphone quality.
- CLI may be difficult for beginners.
- GUI requires more development effort.

Result

The Alert System Application was successfully implemented using CLI, GUI and

VUI. It was observed that:

- GUI provided the best overall user experience.
- CLI was fastest for experienced users.
- VUI offered hands-free interaction but required accurate speech recognition.

Conclusion

This experiment demonstrates the practical implementation of different user interfaces for the same application.

It proves that:

- Different interfaces suit different users.
- GUI provides better usability for general users.
- CLI offers speed and control.
- VUI provides modern, hands-free interaction.

Thus, interface selection plays a critical role in user satisfaction and system usability.